Chapter 6. Dates and Times

6.0 Introduction

Developers suffered for a decade and a half under the inconsistencies and ambiguities of the `Date` class from Java 1.0 and its replacement wannabe, the `Calendar` class from Java 1.1. Several alternative `Date` replacement packages emerged, including the simple and sensible Date4J and the more comprehensive Joda-Time package. Java 8 introduced a new, consistent, and well-thought-out package for date and time handling under the aegis of the Java Community Process, JSR-310,[1] shepherded by developer Stephen Colebourne. Stephen says that "JSR-310 started from scratch, but with an API 'inspired by Joda-Time,'" his earlier package, with several important design changes.[2] This package is biased toward ISO 8601 dates; the default format is, for example, 2015-10-23T10:22:45. But it can, of course, work with other calendar schemes.

One of the key benefits of the new API is that it provides *useful operations* such as adding/subtracting dates/times. Much time was wasted by developers reimplementing these useful operations for the old APIs. With the new APIs, one can use the built-in functionality. That said, millions of lines of code are based on the old APIs, so we'll review them briefly and then consider interfacing the new API to legacy code, in the final recipe of this chapter, Recipe 6.9.

Another advantage of the new API is that almost all objects are immutable and thus thread-safe. This can be of considerable benefit as we move headlong into the massively parallel era.

Because there are no `set` methods, and thus the getter method paradigm doesn't always make sense, the API provides a series of new methods to replace such methods, listed in Table 6-1.

Table 6-1. New date/time API: common methods

Name	Description
`at`	Combines with another object
format	Use provided formatter to produce a formatted string
`from`	Factory: convert input parameters to instance of target
get	Retrieve one field from the instance
`is...`	Examine the state of the given object
minus	Return a copy with the given amount subtracted
`now`	BuilderFactory: get the current time, date, etc.
of	Factory: create new method by parsing inputs
`parse`	Factory: parse single input string to produce instance of target
plus	Return a copy with the given amount added
`to`	Return a copy of this object converted to another type
with	Return a copy with the given field changed; replaces `set` methods

The JSR-310 API specifies a dozen or so main classes. Those representing time are either continuous time or human time. *Continuous time* is based on Unix time, a deeper truth from the dawn of (computer) time, and is represented as a single monotonically increasing number. The time value of 0 in Unix represents the first second of January 1, 1970 UTC—about the time Unix was invented.[3] Each unit of increment thereafter represents one second of time. This has been used as a time base in most operating systems developed since. However, a 32-bit integer representing the number of seconds since 1970 runs out fairly soon—in the year 2038 AD. Most Unix systems have, in the aftermath of the Y2K frenzy, quietly and well in advance, headed off the possibility of a Y2038 frenzy by converting the time value from a 32-bit quantity to a 64-bit quantity.

Java also used this time base, but used 64 bits and stored its time in milliseconds, because a 64-bit time in milliseconds since 1970 will not overflow until quite a few years into the future (keep this date open in your calendar—August 17, 292,278,994 AD). Here is a calculation that shows how I got that date:

```
Date endOfTime = new Date(Long.MAX_VALUE);
System.out.println("Java8 time overflows on " + endOfTime);
```

The new date/time API is in five packages, as shown in Table 6-2; as usual, the top-level package contains the most commonly used pieces.

Table 6-2. New date/time API: packages

Name	Description
java.time	Common classes for dates, times, instants, and durations
java.time.chrono	API for non-ISO calendar systems
java.time.format	Formatting classes (see Recipe 6.2)
java.time.temporal	Date and time access using fields, units, and adjusters
java.time.zone	Support for time zones and their rules

The basic java.time package contains a dozen or so classes, as well as a couple of enums and one general-purpose exception (shown in Tables 6-3, 6-4, and 6-5).

Table 6-3. New date/time API: basics

Class	Description
Clock	Replaceable factory for getting current time
Instant	A point in time since January 1, 1970, expressed in nanoseconds
Duration	A length of time, also expressed in nanoseconds

Human time represents times and dates as we use them in our everyday life. These classes are listed in Table 6-4.

Table 6-4. New date/time API: human time

Class	Description
`DayOfWeek`	A day of the week (e.g., Tuesday)
`LocalDate`	A bare date (day, month, and year) with no adjustments
`LocalTime`	A bare time (hour, minute, seconds) with no adjustments
`LocalDateTime`	The combination of the preceding classes
`Month`	Month-of-year (e.g., January)
`MonthDay`	Month and day (e.g., 02-29)
`OffsetTime`	A time of day with a time zone offset like -04:00, with no date or zone
`OffsetDateTime`	A date and time with a time zone offset like -04:00, with no time zone
`Period`	A descriptive amount of time, such as "2 months and 3 days," stored in a compact string
`Year`	A year by itself
`YearMonth`	A year and month
`ZonedDateTime`	The date and time with a time zone and an offset

Almost all the top-level classes directly extend `java.lang.Object` and are held to consistency by a variety of interfaces, which are declared in the subpackages. The date and time classes mostly implement `Comparable`, which makes sense.

Table 6-5 shows the two time-zone-specific classes used with `ZonedDateTime`, `OffsetDateTime`, and `OffsetTime`.

Table 6-5. New date/time API: support

Class	Description
`ZoneId`	Defines a time zone such as *Canada/Eastern* and its conversion rules
`ZoneOffset`	A time offset from UTC (hours, minutes, seconds)

The new API is a *fluent API*, in which most operations return the object they have operated upon, so that you can chain multiple calls without the need for tedious and annoying temporary variables:

```
LocalTime time = LocalTime.now().minusHours(5); // the time 5 hours ago
```

This results in a more natural and convenient coding style, in my opinion. You can always write code with lots of temporary variables if you want; you're the one who will have to read through it later.

6.1 Finding Today's Date

Problem

You want to find today's date and/or time.

Solution

Invoke the appropriate builder to obtain a `LocalDate`, `LocalTime`, or `LocalDateTime` object. Call its `toString()` method if you want it as a `String`, e.g., to display.

Discussion

These classes do not provide public constructors, so you will need to call one of the factory methods to get an instance. These classes each provide a `now()` method, which does what its name implies. The `CurrentDateTime` demo program shows simple use of all three:

```
public class CurrentDateTime {
  public static void main(String[] args) {
    LocalDate dNow = LocalDate.now();
    System.out.println(dNow);
    LocalTime tNow = LocalTime.now();
    System.out.println(tNow);
    LocalDateTime now = LocalDateTime.now();
    System.out.println(now);
  }
}
```

Running it produces this output:

```
2013-10-28
22:23:55.641
2013-10-28T22:23:55.642
```

The formatting is nothing spectacular, but it's adequate. Fancier formatting is dealt with in Recipe 6.2.

While this works, in full-scale applications, it's recommended to pass a `Clock` instance into all the `now()` methods, to make testing easier. `Clock` is a factory object that is used internally to find the current time. In testing, you often want to have a known date or time used so you can compare against known output. The `Clock` class makes this easy. Example 6-1 uses a `Clock` and allows replacing the default `Clock` by calling a setter. Alternatively you could use a dependency injection framework like CDI or Spring to provide the correct version of the `Clock` class.

Example 6-1. main/src/main/java/datetime/TestableDateTime.java

```
/**
 * TestableDateTime allows test code to plug in a fixed-value
 * clock for e.g., unit testing.
 */
public class TestableDateTime {
  private static Clock clock = Clock.systemDefaultZone();
  public static void main(String[] args) {
    System.out.println("It is now " + LocalDateTime.now(clock));
  }
  public static void setClock(Clock clock) {
    TestableDateTime.clock = clock;
  }
}
```

In normal operation this would get the current date and time. In testing you would call the `setClock()` method with a `Clock` instance obtained from the static method `Clock.fixed(Instant fixedInstant, ZoneId zone)`, passing in the time that your testing code expects. The fixed clock does not tick, so don't worry about the milliseconds between setting the clock to fixed and the invocation of your tests.

6.2 Formatting Dates and Times

Problem

You want to provide better formatting for date and time objects.

Solution

Use `java.time.format.DateTimeFormatter`.

Discussion

The `DateTimeFormatter` class allows you to format dates and times. Its `format()` methods will accept any `TemporalAccessor`, which includes `LocalDate`, `LocalDateTime`, `LocalTime`, `ZonedDateTime`, and a dozen others, including date representations from other calendar styles than the ISO 8601 calendar. There are three categories of methods for getting a `DateTimeFormatter` instance from the class:

- Predefined formatter instances
- Localized formatter instances
- Pattern-based formatting

Predefined formatter instances

There are quite a few predefined instances, as shown in Table 6-6.

Table 6-6. Predefined `DateTimeFormatter` instances

Name	Meaning	Example
BASIC_ISO_DATE	Basic ISO date	*20301111*
ISO_LOCAL_DATE	ISO Local Date	*2030-11-11*
ISO_OFFSET_DATE	ISO Date with offset	*2030-11-11+11:00*
ISO_DATE	ISO Date with or without offset	*2030-11-11+01:00*; *2030-11-11*
ISO_LOCAL_TIME	Time without offset	*11:11:30*
ISO_OFFSET_TIME	Time with offset	*11:11:30+01:00*
ISO_TIME	Time with or without offset	*11:11:30+01:00*; *11:11:00*
ISO_LOCAL_DATE_TIME	ISO Local Date and Time	*2030-11-13T11:11:00*
ISO_OFFSET_DATE_TIME	Date Time with Offset	*2030-12-03T11:11:00+01:00*
ISO_ZONED_DATE_TIME	Zoned Date Time	*2030-12-03T11:11:00+01:00[Europe/Paris]*
ISO_DATE_TIME	Date and time with ZoneId	*2030-12-03T11:11:00+01:00[Europe/Paris]*
ISO_ORDINAL_DATE	Year and day of year	*2012-337*
ISO_WEEK_DATE	Year and Week	*2012-W48-6*
ISO_INSTANT	Date and Time of an Instant	*2030-12-03T11:11:00Z*
RFC_1123_DATE_TIME	RFC 1123/RFC 822	*Tue, 3 Jun 2030 11:11:00 GMT*

These can all be used as formatters, as in this example:

```
System.out.println(
        DateTimeFormatter.ISO_LOCAL_DATE_TIME.format(
                LocalDateTime.now()));
2024-10-16T14:33:57.810557801
```

Localized DateTimeFormatter instances

You can get a localized formatter with one of the following static methods. If you are changing the default `Locale`, do so before calling these methods.

ofLocalizedDate(dateStyle)

Get a formatter with date style from the locale, e.g., *2030-11-11*

ofLocalizedTime(timeStyle)

Get a formatter with time style from the locale, e.g., *11:11:00*

ofLocalizedDateTime(dateTimeStyle)

Get a formatter with a style for date and time from the locale, e.g., *3 Jun 2008 11:05:30*

ofLocalizedDateTime(dateStyle,timeStyle)

Get a formatter with date and time styles from the locale, e.g., *3 Jun 2008 11:05*

These take one or two `style` arguments from the `FormatStyle` enum, which has four values: `FULL`, `LONG`, `MEDIUM`, and `SHORT`. Some examples of this are in Example 6-2.

Example 6-2. main/src/main/java/datetime/DateTimeFormatterLocalized.java

```
public class DateTimeFormatterLocalized {

 public static void main(String[] args) {

   var dt = ZonedDateTime.now();

   for (Locale l :
    List.of(Locale.CANADA, Locale.FRANCE, Locale.UK, Locale.TAIWAN)) {
    Locale.setDefault(l);
    DateTimeFormatter f =
     DateTimeFormatter.ofLocalizedDateTime(FormatStyle.MEDIUM);
    System.out.printf(
     "In Locale %s, date is %s\n",Locale.getDefault(), f.format(dt));
   }
 }
}
```

Running this shows the current date and time in several different locale settings:

```
In Locale en_CA, date is Oct 16, 2024, 5:25:14 p.m.
In Locale fr_FR, date is 16 oct. 2024, 17:25:14
In Locale en_GB, date is 16 Oct 2024, 17:25:14
In Locale zh_TW, date is 2024年10月16日 下午5:25:14
```

The Taiwan output is only garbled because of font issues.

Any `DateTimeFormatter` instance can also be (re)localized (see Recipe 3.12) after construction. Whether you used one of the `ofLocalized...()` factory methods or not, you can always get an instance configured for a new locale by calling `withLocale()` on the instance; as the name hints, you get back a new instance (remember that all classes in `java.time` create immutable objects):

```
void main() {
  var df = DateTimeFormatter.ofLocalizedDateTime(FormatStyle.MEDIUM);
  var now = LocalDateTime.now();
  System.out.println(df.format(now));
  var newdf = df.withLocale(Locale.UK); // new instance with updated Locale info
  System.out.println(newdf.format(now));
}
```

The output shows a given date and time formatted in my default locale and in the UK locale:

```
Oct 16, 2024, 5:50:00 PM
16 Oct 2024, 17:50:00
```

Pattern-based formatting

Finally, if you need the ultimate in flexibility or just don't want to use one of the provided predefined formats, you can define your own by using `DateTimeFormatter.ofPattern(String pattern)`. The `pattern` string can contain any characters, but almost every letter of the alphabet has been defined to mean something, in addition to the obvious `Y`, `M`, `D`, `h`, `m`, and `s`. In addition, the quote character and square bracket characters are defined, and all letters in the Roman alphabet as well as the sharp sign (#) and curly braces are reserved.

As is common with date formatting languages, the number of repetitions of a letter in the pattern gives a clue to its intended length of detail. Thus, for example, `MMM` gives `Jan`, whereas `MMMM` gives `January`.

NOTE

Don't confuse this with `java.text.DateFormat`, which works on the obsolete `java.util.Date` class.

Table 6-7 is an attempt at a complete list of the formatting characters, adapted from the Javadoc for the `DateTimeFormatter` class.

Table 6-7. `DateTimeFormatter` format characters

Symbol	Meaning	Presentation	Examples
G	Era	Text	AD; Anno Domini
y	Year of era	Year	2004; 04
u	Year of era	Year	See the following note.
D	Day of year	Number	189
M/L	Month of year	Number/text	7; 07; Jul; July; J
d	Day of month	Number	10
Q/q	Quarter of year	Number/text	3; 03; Q3, 3rd quarter
Y	Week-based year	Year	1996; 96
w	Week of week-based year	Number	27
W	Week of month	Number	4
e/c	Localized day of week	Number/text	2; 02; Tue; Tuesday; T
E	Day of week	Text	Tue; Tuesday; T
F	Week of month	Number	3
a	a.m./p.m. of day	Text	PM
h	Clock hour of a.m./p.m. (1-12)	Number	12
K	Hour of a.m./p.m. (0-11)	Number	0
k	Clock hour of a.m./p.m. (1-24)	Number	0
H	Hour of day (0-23)	Number	0
m	Minute of hour	Number	30
s	Second of minute	Number	55
S	Fraction of second	Fraction	978
A	Millisecond of day	Number	1234

Symbol	Meaning	Presentation	Examples
n	Nanosecond of second	Number	987654321
N	Nanosecond of day	Number	1234000000
V	Time zone ID	Zone-id	America/Los_Angeles; Z; -08:30
z	Time zone name	Zone-name	Pacific Standard Time; PST
X	Zone offset *Z* for zero	Offset-X	Z; -08; -0830; -08:30; -083015; -08:30:15;
x	Zone offset	Offset-x	+0000; -08; -0830; -08:30; -083015; -08:30:15;
Z	Zone offset	Offset-Z	+0000; -0800; -08:00;
O	Localized zone offset	Offset-O	GMT+8; GMT+08:00; UTC-08:00;
p	Pad next	Pad modifier	1

TIP

Unlike `printf`-style formatting, do not precede these format specifiers with the `%` symbol.

Some examples are shown in Example 6-3.

Example 6-3. main/src/main/java/datetime/DateTimeFormatterDemo.java

```
public class DateTimeFormatterDemo {
  public static void main(String[] args) {

    // Format a date ISO8601-like but with slashes instead of dashes
    DateTimeFormatter df = DateTimeFormatter.ofPattern("yyyy/LL/dd");
    System.out.println(df.format(LocalDate.now()));

    // Format a ZonedDateTime without timezone information
    DateTimeFormatter nTZ =
      DateTimeFormatter.ofPattern("d MMMM, yyyy h:mm a");
    System.out.println(ZonedDateTime.now().format(nTZ));
  }
}
```

Where there are two letters with a slash between, the first is the "standard" form and the second is the "standalone" form, e.g, when displaying that field by itself rather than as part of a date/time string. With one or two copies of the character, it prints numerically, with a leading zero if needed. With three, it prints a short form, with four, a longer form. With five, it prints the "abbreviated form," which is the shortest possible, e.g., `O` for October. In practice there seems little difference, as shown by Example 6-4.

Example 6-4. main/src/main/java/datetime/DateTimeFormatterLM.java

```
/// Try to show the difference between "standard" and "standalone"
/// formats such as "M/L" in [java.time.format.DateTimeFormatter].
///
public class DateTimeFormatterLM {
  public static void main(String[] args) {
    System.out.println("L       M");
    var then = LocalDate.of(2222,2,2);
    IntStream.rangeClosed(1, 5).forEach( i -> {
      var str = "L".repeat(i) + " " + "M".repeat(i);
      var fmt = DateTimeFormatter.ofPattern(str);
      System.out.printf("Length %d (%s) gives %s\n", i, str, fmt.format(then));
    });
  }
}
```

Running this program prints the following, showing all five formats:

```
$ java src/main/java/datetime/DateTimeFormatterLM.java
L       M
Length 1 (L M) gives 2 2
Length 2 (LL MM) gives 02 02
Length 3 (LLL MMM) gives Feb Feb
Length 4 (LLLL MMMM) gives February February
Length 5 (LLLLL MMMMM) gives F F
```

NOTE

`y` and `u` work the same for AD years; however, for a year of 3 BC, the `y` pattern returns 3, whereas the `u` pattern returns –2 (a.k.a. proleptic year).

6.3 Converting Among Dates/Times and Epoch Seconds

Problem

You need to convert among dates/times, epoch seconds, or some other numeric value.

Solution

Use the appropriate date/time factory or retrieval methods.

Discussion

The epoch is the beginning of time as far as modern operating systems go. Unix time, and some versions of Windows time, count off inexorably the seconds since the epoch. When Ken Thompson and Dennis Ritchie came up with this format in 1970, seconds seemed like a fine measure, and 32 bits' worth of seconds seemed nearly infinite. On operating systems that store the epoch in a 32-bit integer, however, time is running out. Older versions of most operating systems stored this as a 32-bit signed integer, which unfortunately will overflow on January 18, 2038, at 22:14:07 Eastern time.

When Java first came out, it featured a method called `System.currentTimeMillis()`, which presented epoch seconds with millisecond accuracy.[4]

Any of these epoch-related numbers can be converted into, or obtained from, a local date/time. Other numbers can also be used, such as integer years, months, and days. As usual, there are factory methods that create new objects where a change is requested. Here is a program that shows some of these conversions in action:

main/src/main/java/datetime/DateConversions.java

```
// Convert "a billion seconds of Unix" to a local date/time
Instant epochSec = Instant.ofEpochSecond(1000000000L);
ZoneId zId = ZoneId.systemDefault();
ZonedDateTime then = ZonedDateTime.ofInstant(epochSec, zId);
System.out.println("The epoch was a billion seconds old on " + then);

// Convert a date/time to Epoch seconds
long epochSecond = ZonedDateTime.now().toInstant().getEpochSecond();
```

```
    System.out.println("Current epoch seconds = " + epochSecond);

    ZonedDateTime here = ZonedDateTime.now();
    ZonedDateTime there = here.withZoneSameInstant(ZoneId.of("Canada/Pacific"));
    System.out.printf("When it's %s here, it's %s in Vancouver%n", here, there);
```

6.4 Parsing Strings into Dates

Problem

You need to convert user input into `java.time` objects.

Solution

Use a `parse()` method.

Discussion

Many of the date/time classes have a `parse()` factory method, which tries to parse a string into an object of that class. For example, `LocalDate.parse(String)` returns a `LocalDate` object for the date given in the input `String`, as shown in Example 6-5.

Example 6-5. main/src/main/java/datetime/DateParse.java

```
public class DateParse {
  public static void main(String[] args) {

    String armisticeDateString = "1918-11-11";
    LocalDate armisticeDate = LocalDate.parse(armisticeDateString);
    System.out.println("Date: " + armisticeDate);

    String armisticeDateTimeString = "1918-11-11T11:00";
    LocalDateTime armisticeDateTime =
      LocalDateTime.parse(armisticeDateTimeString);
    System.out.println("Date/Time: " + armisticeDateTime);
```

TIP

The `parse()` routines in these classes will throw a `DateTimexParseException` if the input string cannot be parsed according to the (default or specified) pattern.

As you probably expect by now, the default format is the ISO 8601 date format. However, we often have to deal with dates in other formats. There is a series of predefined formatters defined as fields in the `DateTimeFormatter` class, such as `BASIC_ISO_DATE`, `ISO_LOCAL_DATE_TIME`, `ISO_LOCAL_TIME`, and more—see Table 6-1 and also the Javadoc for the complete list. Additionally, you might want to define your own specific format. For this, the `DateTimeFormatter` allows you to specify a particular pattern. For example, `"dd MMM uuuu"` represents the day of the month (two digits), three letters of the name of the month (Jan, Feb, Mar, ...), and a four-digit year:

```
    // DateParse.java (continued)
// You can also parse with any DateTimeFormatter that matches your input
DateTimeFormatter df = DateTimeFormatter.ofPattern("dd MMM uuuu");
String anotherDate = "27 Jan 2027";
LocalDate random = LocalDate.parse(anotherDate, df);
System.out.println(anotherDate + " parses as " + random);
```

The `DateTimeFormatter` object is bidirectional; it can both parse input and format output. We could add this line to the `DateParse` example:

```
// DateParse.java (continued)
System.out.println(armisticeDate + " formats as " + df.format(armisticeDate));
```

When we run the program, we see the output as follows:

```
Date: 1918-11-11
Date/Time: 1918-11-11T11:00
27 Jan 2027 parses as 2027-01-27
1918-11-11 formats as 11 Nov 1918
```

INSTANTS, DURATIONS, AND PERIODS

Timekeeping is inherently complex, as we live on a large globe—the Earth—that spins at a rate of *roughly* 24 hours (86,400 seconds) per day. Minor adjustments have to be made from time to time, as this planetary day is somewhat irregular. If this is of interest, please refer to the Javadoc for `Instant`. When reading this and related documents, you will come across the uncommon word *proleptic Gregorian calendar*. This simply means the modern Gregorian/ISO 8601 calendar extrapolated into the past, ignoring details such as the *week and a half skipped* at many different times in different countries when converting from the Julian calendar.[5]

At least three classes represent particular points of interest. An `Instant` has about the same meaning as in everyday speech, a (very) specific point in time. A `Period` has one of the same meanings as in English, a number of days (represented as years, months, and days). A `Duration` is, as in the common tongue, a number of seconds (represented as hours, minutes, and seconds). `Period` and `Duration` therefore both represent *spans of time*. The `Instant` class represents a particular point in time, measured in seconds with nanosecond precision.

Both `Instant` and `Duration` (but not `Period`) store their time internally in two parts:

- A `long` representing the number of seconds since the beginning of (Unix/Java) time, January 1st, 1970, at 00:00 UTC
- An `int` representing the number of nanoseconds, which ranges from 0 to 999,999,999

Interestingly, parts of a period can be negative, and will be treated individually:

```
LocalDate.now();
$18 ==> 2024-05-04
jshell> LocalDate.now().plus(Period.of(4,-1,12));
$19 ==> 2028-04-16
```

The years field was added; the month was negative so it was subtracted, and the days value was positive so it was added. I'm not sure of the use case for this, but there it is.

A `Duration` represents a span of time that is usually a day or less. It is represented as a string in the form `PTnHnMn.sS` where each combination of a number *n* and the *following* character represents that number of hours, minutes, and seconds (with optional milli- and nanoseconds). It can be created using various factory and plus/minus methods:

```
Duration.ofHours(12).plusHours(4).plusMillis(3).plusMinutes(6);
PT16H6M0.003S
```

There is a `plusNanos()` method but not a `plusMicros()`.

The `Period` class represents the difference between two instants. It's presented as a compact string in the form `PnYnMnDnHnMnS` where each combination of a number *n* and the following character represents that number of years, months, etc. For example, `P5Y2D` means five years and two days. While the `Period` class isn't meant to be human-friendly, there is a formatter for it in *darwinsys-api/src/main/java/com/darwinsys/formatting/DateFormatting.java* and a test program demonstrating its use.

The `Period` and `Duration` objects are similar, though obviously differing in scale. Another difference only shows up when they are used in certain calculations, e.g., when added to a `ZonedDateTime`. Adding a `Duration` would add an exact number of seconds, so that a `Duration` of one day is always exactly 24 hours. A `Period`, on the other hand, would add a "conceptual day," taking account of time differences such as that which occur when entering or leaving Daylight Saving Time.

I did say it was complicated.

The recipes in this section cover some uses of these classes.

6.5 Difference Between Two Dates

Problem

You need to compute the difference between two dates.

Solution

Use the static method `Period.between()` to find the difference between two `LocalDates`.

Discussion

Given two `LocalDate` objects, you can find the difference between them, as a `Period`, simply using the static `Period.between()` method. You can `toString()` the `Period` or, if its default format isn't good enough, format the result yourself:

```
import java.time.LocalDate;
import java.time.Period;

/**
 * Tutorial/Example of LocalDate date difference subtraction
 */
public class DateDiff {

  public static void main(String[] args) {
    /* The date at the end of the last century */
    LocalDate endOf20thCentury = LocalDate.of(2000, 12, 31);
    LocalDate now = LocalDate.now();
    if (now.getYear() > 2100) {
      System.out.println("The 21st century is over!");
      return;
    }

    Period diff = Period.between(endOf20thCentury, now);

    System.out.printf("The 21st century (up to %s) is %s old%n", now, diff);
    System.out.printf(
        "The 21st century is %d years, %d months and %d days old%n",
        diff.getYears(), diff.getMonths(), diff.getDays());
  }
}
```

I reran this code near the end of October 2024; the 20th century AD ended at the end of 2000 (not the end of 1999!), so the value should be about 23 $^{10}/_{12}$ years, and it was:

```
$ java datetime.DateDiff
The 21st century (up to 2024-10-21) is P23Y9M21D old
The 21st century is 23 years, 9 months and 21 days old
```

While the `Period` class isn't meant to be human-friendly, pulling out the fields as was done here is tedious.

Because of the API's regularity, you can use the same technique with `LocalTime` or `LocalDateTime`.

There is also `ChronoUnit`, which has numerous range values such as `DAYS`, `HOURS`, `MINUTES`, etc. (actually ranging from `NANOS` for nanoseconds up to `MILLENIA`, `ERAS`, and even `FOREVER`). If you want difference information in a certain unit:

```
jshell> import java.time.temporal.*;

jshell> ChronoUnit.DAYS.between(LocalDate.now(), LocalDate.parse("2033-03-31"));
$1 ==> 3253

jshell> ChronoUnit.DECADES.between(LocalDate.of(1970,01,01),
  LocalDate.now());
$2 ==> 5
```

Unix has completed five decades of active service and is part way into its sixth decade!

See Also

A higher-level way of formatting date/time values is discussed in Recipe 6.2.

6.6 Adding to or Subtracting from a Date

Problem

You need to add or subtract a fixed period to or from a date.

Solution

Create a past or future date by using a method such as `LocalDate.plus(Period.ofDays(N));`.

Discussion

As we've seen, the `Period` class represents a length of time, such as a number of days. `LocalDate` and friends offer a series of `plus()` and `minus()` methods to add or subtract a `Period` or other time-related object, e.g., `plusDays(5)`. `Period` offers factory meth-

ods such as `ofDays()`. The following code computes what the date will be 700 days from now:

```
import java.time.LocalDate;
import java.time.Period;

/** DateAdd -- compute the difference between two dates
 * (e.g., today and 700 days from now).
 */
public class DateAdd {
  public static void main(String[] av) {
    /* Today's date */
    LocalDate now =  LocalDate.now();

    LocalDate then1 = now.plusDays(700);
    Period p = Period.ofDays(700);
    LocalDate then2 = now.plus(p);

    System.out.printf("Seven hundred days from %s is %s or %s%n",
      now, then1, then2);
  }
}
```

Running this program reports the current date and time and what the date and time will be 700 days from now:

```
Seven hundred days from 2024-05-03 is 2026-04-03
```

6.7 Calculating Recurring Events

Problem

You need to deal with recurring dates, for example, the third Wednesday of every month.

Solution

Use the `TemporalAdjusters` class.

Discussion

The `TemporalAdjuster` interface and the `TemporalAdjusters` factory class provide most of what you need for recurring events. There are many interesting and powerful adjusters available, shown in Table 6-8, and you can, of course, develop your own.

Table 6-8. New date/time API: `TemporalAdjusters` factory methods

Method signature
`public static TemporalAdjuster firstDayOfMonth();`
`public static TemporalAdjuster lastDayOfMonth();`
`public static TemporalAdjuster firstDayOfNextMonth();`
`public static TemporalAdjuster firstDayOfYear();`
`public static TemporalAdjuster lastDayOfYear();`
`public static TemporalAdjuster firstDayOfNextYear();`
`public static TemporalAdjuster firstInMonth(java.time.DayOfWeek);`
`public static TemporalAdjuster lastInMonth(java.time.DayOfWeek);`
`public static TemporalAdjuster dayOfWeekInMonth(int, java.time.DayOfWeek);`
`public static TemporalAdjuster next(java.time.DayOfWeek);`
`public static TemporalAdjuster nextOrSame(java.time.DayOfWeek);`
`public static TemporalAdjuster previous(java.time.DayOfWeek);`
`public static TemporalAdjuster previousOrSame(java.time.DayOfWeek);`
`public static TemporalAdjuster ofDateAdjuster( java.util.function.UnaryOperator<java.time.LocalDate>);`

The names of most of these tell you directly what they do. The last one will make sense after reading about functional interfaces such as `UnaryOperator` in Chapter 9.

These are used with the `with()` method of a date/time object. For example, the GTABUG group (*https://gtabug.org*) meets on the third Wednesday of every month. I have a `RecurringEventDatePicker` class in the darwinsys-api library; the core of it started as the method `getMeetingDateInMonth(LocalDate dateContainingMonth)`, which in our case picks the third Wednesday of a given month (given that *dayOfWeek* and *weekOfMonth*

are both set in the constructor). The file *MeetingDates.java* shown in Example 6-6 is a simplified version of this code. This uses the `firstInMonth()` factory method to get a temporal adjuster, then adds the number of weeks to get the Wednesday in the correct week. Its only complexity is that, for the current month, the meeting might be before or after today's date, so we adjust accordingly.

Example 6-6. main/src/main/java/datetime/MeetingDates.java

```
public static LocalDate getNextMeeting(
    int weekOfMonth,
    DayOfWeek dayOfWeek,
    int meetingsAway) {
  LocalDate now = LocalDate.now();
  LocalDate thisMeeting = now.with(
    TemporalAdjusters.dayOfWeekInMonth(weekOfMonth,dayOfWeek));
  // Has the meeting already happened this month?
  if (thisMeeting.isBefore(now)) {
    // start from next month
    meetingsAway++;
  }
  if (meetingsAway > 0) {
    thisMeeting = thisMeeting.plusMonths(meetingsAway).
      with(TemporalAdjusters.dayOfWeekInMonth(weekOfMonth,dayOfWeek));
  }
  return thisMeeting;
}
```

The `main` method that invokes it is shown in Example 6-7.

Example 6-7. main/src/main/java/datetime/MeetingDates.java

```
public static void main(String[] args) {
  DateTimeFormatter dfm = DateTimeFormatter.ofPattern("MMMM dd, yyyy");
  for (int monthAway = 0; monthAway <= 2; monthAway++) {
    LocalDate dt = getNextMeeting(weekOfMonth, dayOfWeek, monthAway);
    System.out.println(dt.format(dfm));
  }
}
```

When visiting this site in Summer 2024, you might have seen something like this:

```
$ java MeetingDates.java
July 17, 2024
August 21, 2024
September 18, 2024
```

6.8 Computing Dates Involving Time Zones

Problem

Imagine a problem like "Your kids are traveling on a transatlantic flight from Toronto to London that takes 5 hours 10 minutes from the actual time of departure from YYZ. Your in-laws need one hour to get to LHR and find parking. What time should you phone them to leave for the airport?"

Solution

The solution needs to take account of time zone differences. It can be solved using the `ZonedDateTime` class and methods such as `plus()` and `minus()` from that class.

Discussion

The basic steps are shown in Example 6-8.

Example 6-8. main/src/main/java/datetime/FlightArrivalTimeCalc.java

```
public class FlightArrivalTimeCalc {

 static Duration driveTime = Duration.ofHours(1);

 public static void main(String[] args) {
  LocalDateTime when = null;
  if (args.length == 0) {
   when = LocalDateTime.now();                          ❶
  } else {
   String time = args[0];
   LocalTime localTime = LocalTime.parse(time);
   when = LocalDateTime.of(LocalDate.now(), localTime);          ❶
  }
  calculateArrivalTime(when);
 }
```

```
  public static ZonedDateTime calculateArrivalTime(LocalDateTime takeOffTime) {
    ZoneId torontoZone = ZoneId.of("America/Toronto"),
       londonZone = ZoneId.of("Europe/London");
    ZonedDateTime takeOffTimeZoned =
     ZonedDateTime.of(takeOffTime, torontoZone);                    ❷
    Duration flightTime =
     Duration.ofHours(5).plus(10, ChronoUnit.MINUTES);              ❸
    ZonedDateTime arrivalTimeUnZoned = takeOffTimeZoned.plus(flightTime); ❹
    ZonedDateTime arrivalTimeZoned =
     arrivalTimeUnZoned.toInstant().atZone(londonZone);             ❺
    ZonedDateTime phoneTimeHere = arrivalTimeUnZoned.minus(driveTime);   ❻

    System.out.println("Flight departure time " + takeOffTimeZoned);
    System.out.println("Flight expected length: " + flightTime);
    System.out.println(
     "Flight arrives there at " + arrivalTimeZoned + " London time.");
    System.out.println("You should phone at " + phoneTimeHere + " Toronto time");
    return arrivalTimeZoned;
   }
  }
```

❶ Get the departure time as a `LocalDateTime` (defaulting to `now()` if no arguments passed into `main()`, on the assumption that we run the app when the flight takes off).

❷ Convert departure time to `ZonedDateTime`.

❸ Convert flight time to a `Duration`.

❹ Get the arrival time by adding the departure time to the flight duration.

❺ Convert the arrival time to London time with `atZone()`.

❻ Since the family takes an hour to get to the airport, subtract that from the arrival time. This yields the time when you should phone them.

6.9 Interfacing with Legacy Date and Calendar

Classes

Problem

You need to deal with the old `Date` and `Calendar` classes.

Solution

Assuming you have code using the original `java.util.Date` and `java.util.Calendar`, you can convert values as needed using conversion methods.

Discussion

All the classes and interfaces in the new API were chosen to avoid conflicting with the traditional API. It is thus possible, and will be common for a while, to have imports from both packages in the same code.

To keep the new API clean, most of the necessary conversion routines were *added to the old API*. Table 6-9 summarizes these conversion routines; note that the methods are static if they are shown being invoked with a capitalized class name, otherwise they are instance methods.

Table 6-9. Legacy date/time interchange

Legacy class	Convert to legacy	Convert to modern
`java.util.Date`	`date.from(Instant)`	`Date.toInstant()`
`java.util.Calendar`	`calendar.toInstant()`	-
`java.util.GregorianCalendar`	`GregorianCalendar.from(ZonedDateTime)`	`calendar.toZonedDateTime()`
`java.util.TimeZone`	-	`timeZone.toZoneId()`
`java.text.Format`	`dateTimeFormatter.toFormat()`	n/a

Example 6-9 shows some of these APIs in action.

Example 6-9. main/src/main/java/datetime/LegacyDates.java

```
public class LegacyDates {
  public static void main(String[] args) {

    // There and back again, via Date
    Date legacyDate = new Date();
    System.out.println(legacyDate);

    LocalDateTime newDate =
      LocalDateTime.ofInstant(legacyDate.toInstant(),
      ZoneId.systemDefault());
    System.out.println(newDate);

    Date backAgain =
      Date.from(newDate.atZone(ZoneId.systemDefault()).toInstant());
    System.out.println("Converted back as " + backAgain);

    // And via Calendar
    Calendar c = Calendar.getInstance();
    System.out.println(c);
    LocalDateTime newCal =
      LocalDateTime.ofInstant(c.toInstant(),
      ZoneId.systemDefault());
    System.out.println(newCal);
  }
}
```

You do not have to use these legacy converters; you are free to write your own, though doing so is usually a fool's errand. The file *LegacyDatesDIY.java* in the *javasrc* repository explores this option in the unlikely event you wish to pursue it.

Given the amount of code written before Java 8, it is likely that the legacy `Date` and `Calendar` will be around until the end of Java time.

The new date/time API has many capabilities that we have not explored. Almost enough for a small book on the subject, in fact. Meanwhile, you can study the API details at Oracle.

1 JSR stands for Java Specification Request. The Java Community Process calls standards, both proposed and adopted, JSRs. See the Java Community Process for details.

2 For those with an interest in historical arcana, the differences are documented on his blog.

3 This is late in the day of December 31, 1969, in the Eastern time zone, so when you run across a file with a "last modified" time in that day, it pretty much invariably means that part of the file's meta-data got set to zero by accident.

4 The modern Java API uses a nanosecond-resolution timestamp that is *not* related to the epoch (i.e., can only be compared with the same value later in the same run of the JVM). It can be obtained with a call to `System.nanoTime()`.

5 We can blame the Romans for both calendars; the Julian was decreed by Roman Emperor Julius Caesar (before he failed to beware the ides of March in his own calendar system), and the Gregorian was decreed by Roman Catholic Pope Gregory XIII.

Previous chapter
< 5. Numbers

Next chapter
7. Structuring Data with Java >